Lecture 21

Applying Deep Learning to Your Problems

(Hacker's Guide to DL)

Speaker: Kaiming He

- Deep Learning is about **making things work**
- Applying Deep Learning is ultimately about **building systems**
- Every aspect matters: small subtleties can make **big differences**
- This lecture: provides an **operation manual**

How can we make things work?

- Data
- Model
- Optimizer
- Monitor
- Reproducibility

How can we make things work?

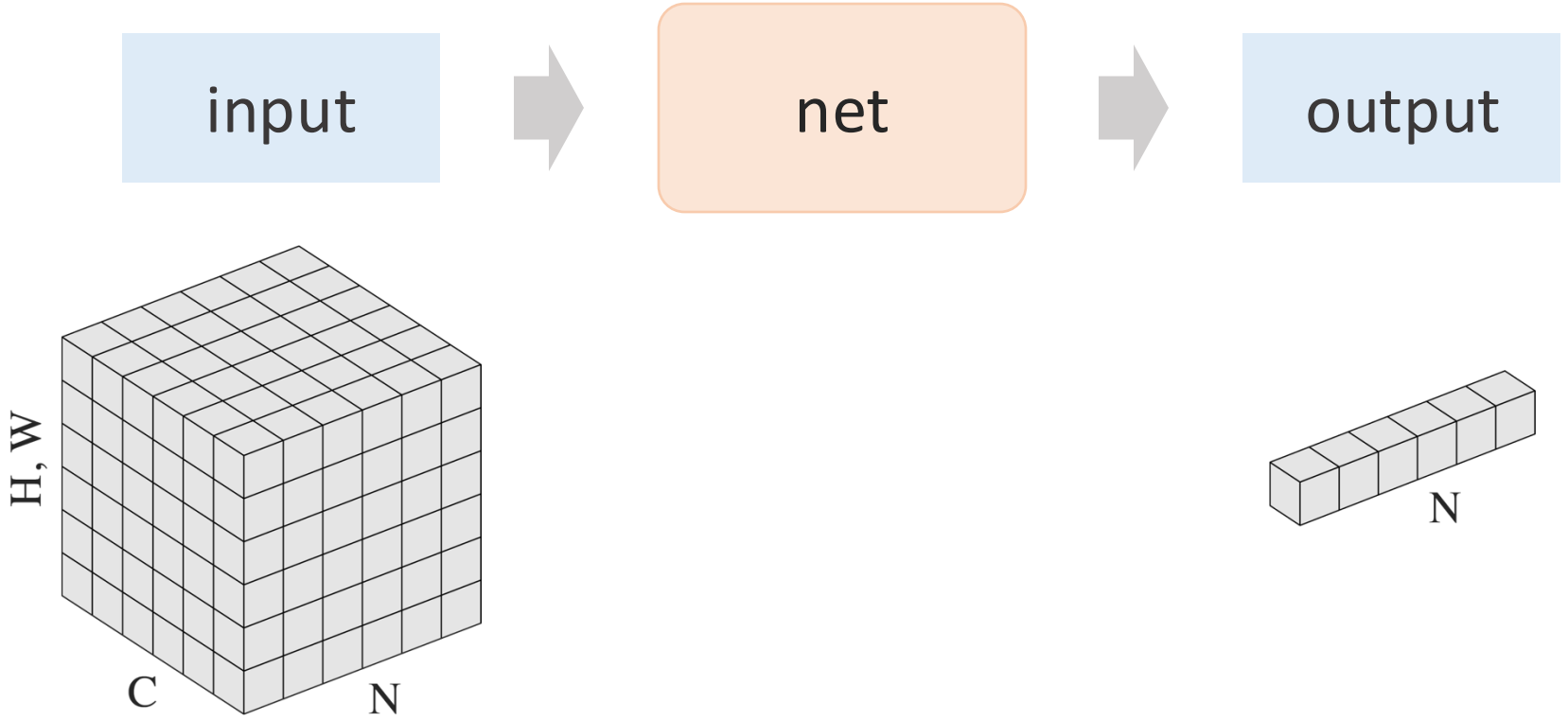
- Data
- Model
- Optimizer
- Monitor
- Reproducibility

Data

- What are the **input** and **output** of your problem?

Data

- What are the **input** and **output** of your problem?
- Mindset: **tensor shapes**



Data

- What are the **input** and **output** of your problem?
- Mindset: **tensor shapes**

example:
image classification

N x C x H x W

N x 1 (class index)

N x K (one-hot)

ImageNet
N x 3 x 224 x 224

CIFAR-10
N x 3 x 32 x 32

ImageNet
N x 1000

CIFAR-10
N x 10

Data

- What are the **input** and **output** of your problem?
- Mindset: **tensor shapes**

example:
image classification

N x C x H x W

N x 1 (class index)

N x K (one-hot)

- Omit batch size for brevity
- Sometimes it matters (e.g., contrastive learning)

Data: input/output shapes - case study

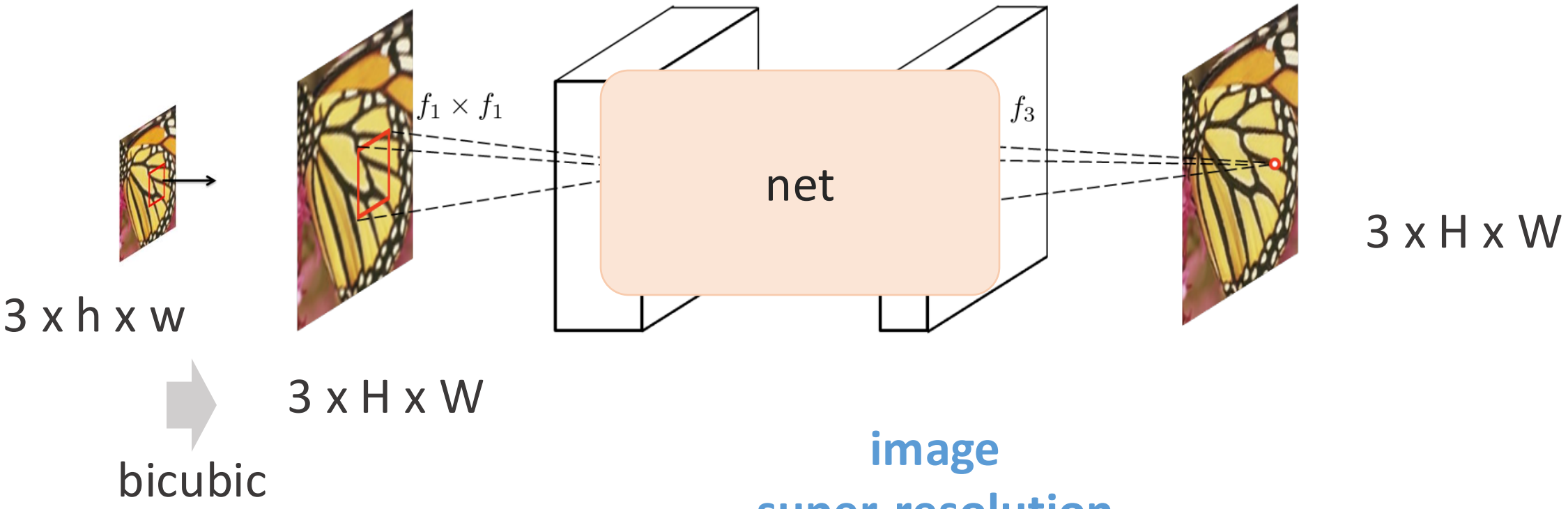
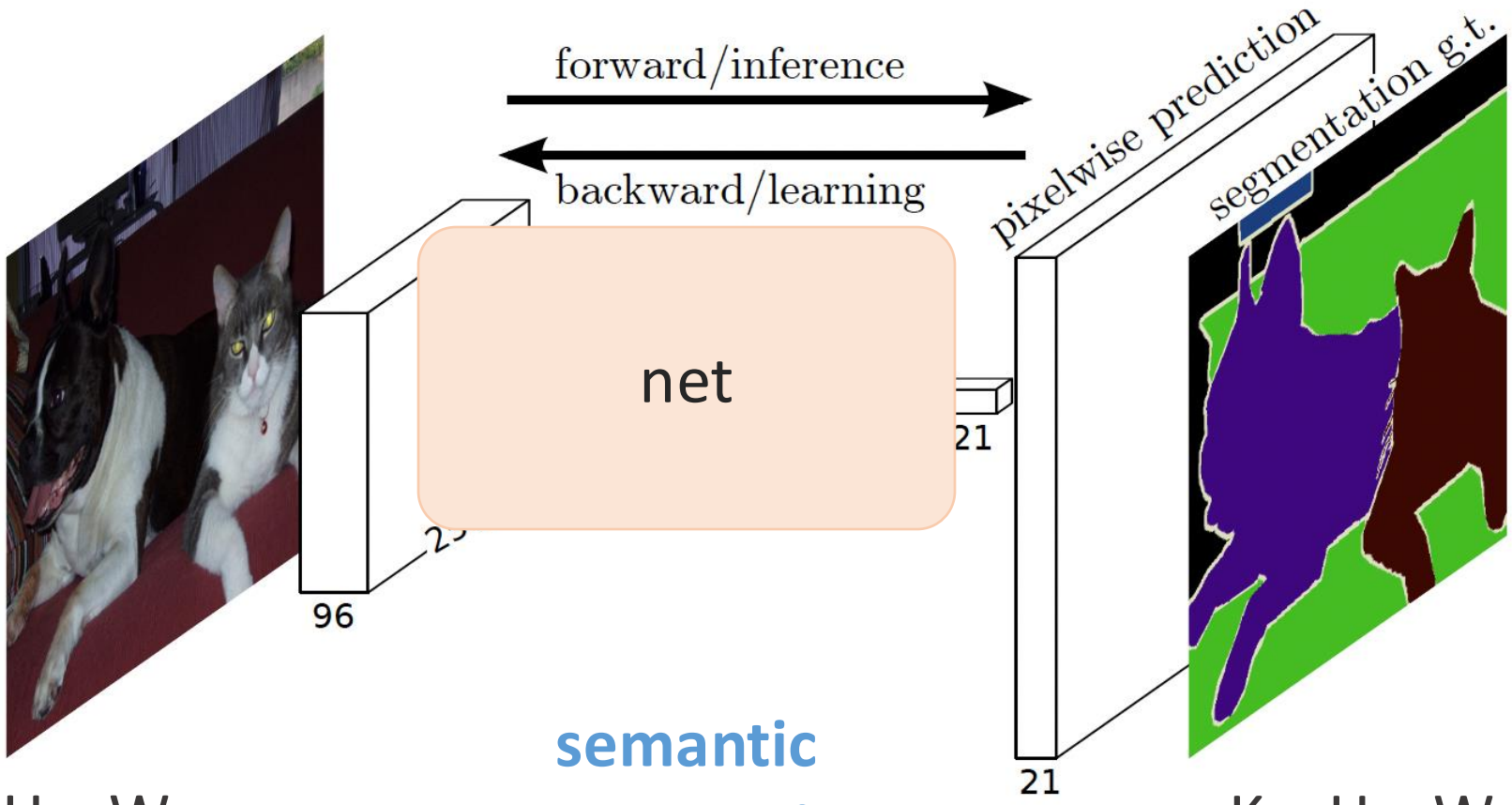


image super-resolution

Dong et al. Learning a Deep Convolutional Network for Image Super-Resolution, ECCV 2014

Data: input/output shapes - case study



3 x H x W

semantic segmentation

K x H x W
(K classes)

Long et al. Fully Convolutional Networks for Semantic Segmentation, CVPR 2015

Data: input/output shapes - case study

net

3 x H x W

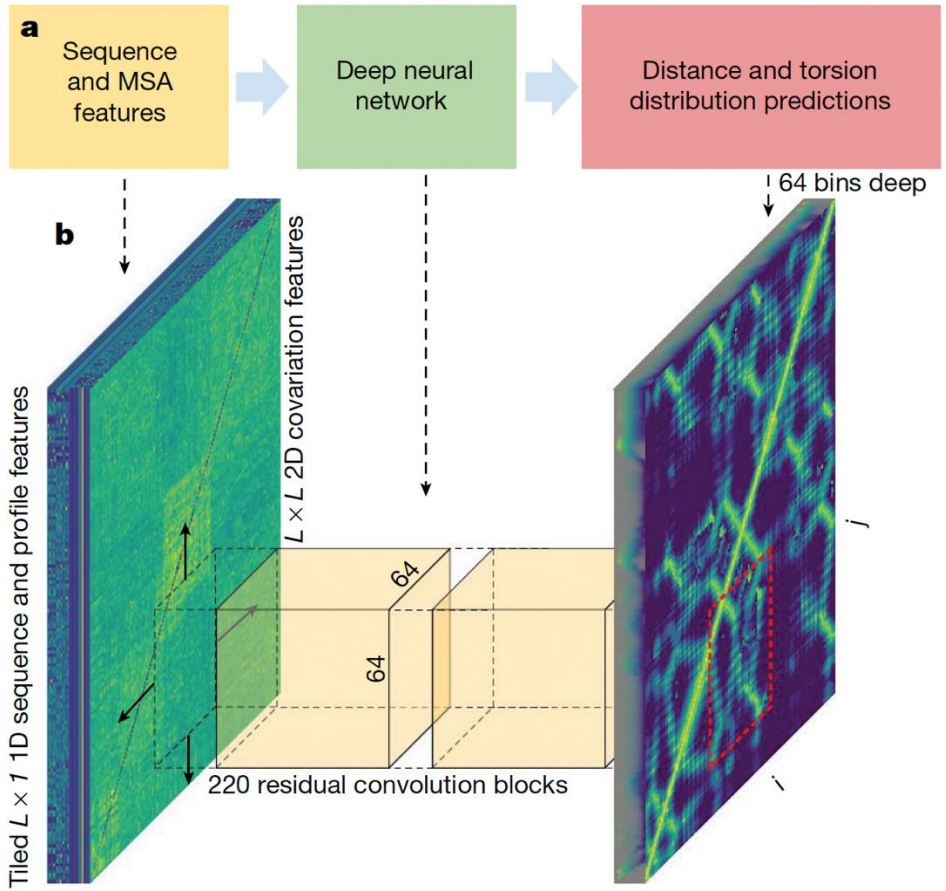
3 x H x W

denoising
(and diffusion)

Data: input/output shapes - case study

L x L x C

- seq len: L
- 1-D tiled: L x L x c_1
- 2-D cov: L x L x c_2



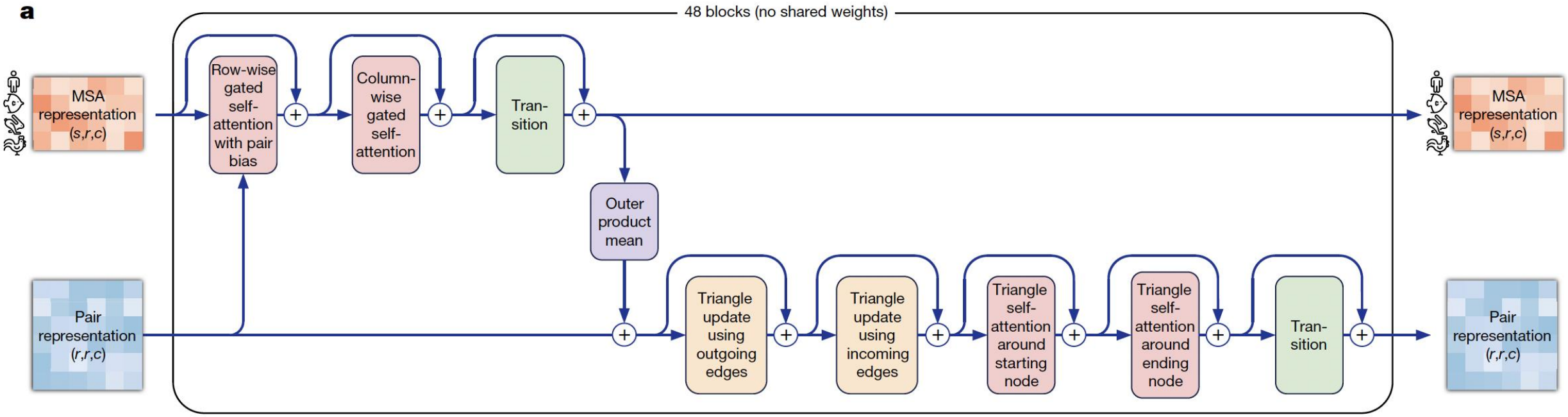
L x L x 64 bins

AlphaFold 1

- AlphaFold 1 is a 2D ConvNet!

Senior et al. Improved protein structure prediction using potentials from deep learning, Nature 2020

Data: input/output shapes - case study



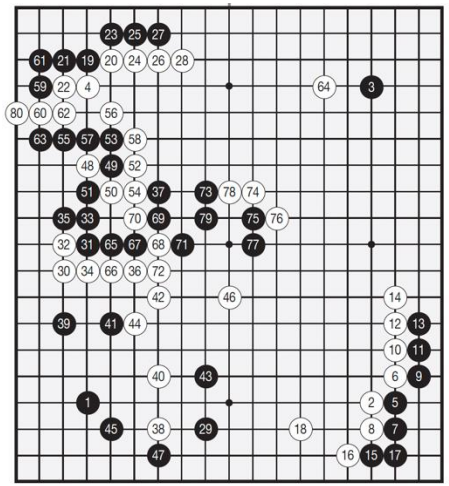
s: # MSA sequences
r: residue length
c: channels

(MSA: multiple sequence alignment)

AlphaFold 2

Jumper et al. Highly accurate protein structure prediction with AlphaFold, Nature 2021

Data: input/output shapes - case study



19 x 19 x C

(C: current/past states)

policy network

362 actions

- 361: 19x19 points
- +1: “pass”

value network

1 scalar value

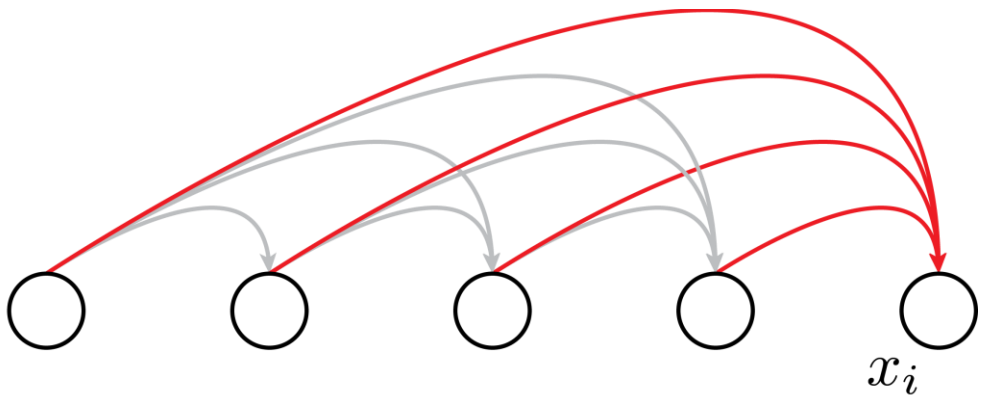
- win probability
- or result: -1 or 1

AlphaGo Zero

Silver et al. Mastering the game of Go without human knowledge, Nature 2017

Data: input/output shapes - case study

$$p(x_i \mid x_1, x_2, \ldots, x_{i-1})$$



for **one** conditional prob

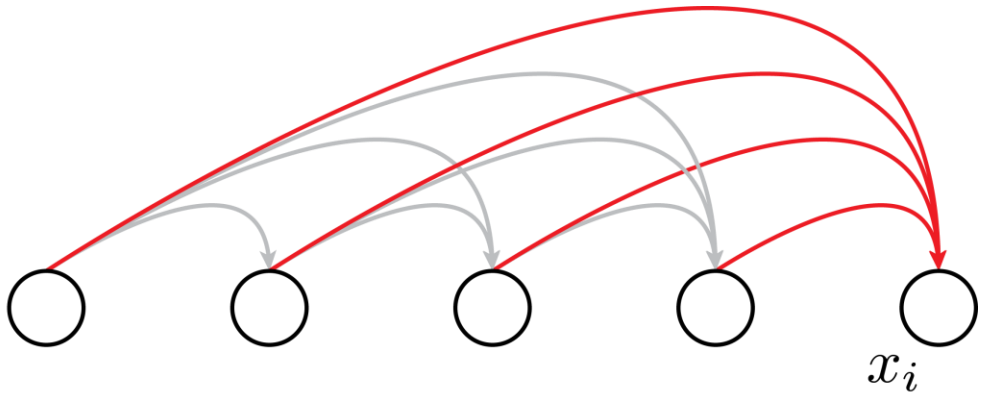
- L: prefix length
- K: vocab size
- input: L x K
- output: K

Autoregression/GPT

Radford et al. Improving Language Understanding by Generative Pre-Training, 2018

Data: input/output shapes - case study

$$
\begin{aligned}
p(x_1, x_2, ..., x_n) &= p(x_1)p(x_2 \mid x_1)...p(x_n \mid x_1, x_2, ..., x_{n-1}) \\
&= \prod_{i=1}^{n} p(x_i \mid x_1, x_2, ..., x_{i-1})
\end{aligned}
$$



for **joint** prob

- L: prefix length
- K: vocab size
- input: L x K
- output: L x K or L x (K - 1)

Autoregression/GPT

Radford et al. Improving Language Understanding by Generative Pre-Training, 2018

Data

- What are the **input** and **output** of your problem?
- Mindset: **tensor shapes**
- **Pre/post-process**: prepare your problem for neural networks

- data augmentation
- resizing
  - crop, trunc, pad, ...
- feature engineering
  - freq, hist, ...
- tokenization

- heuristics
  - non-maximum suppression,
  - bipartite matching, ...
- de-tokenization
- project to feasible space
- MCTS, RL, ...

Data

Estimate your problem scale

- Total **data dimensionality**?
  - # pixels: (# images) x H x W x C
  - # tokens: (# sequence) x (avg. tokens per seq)
- Total **information**?
  - roughly the disk size after compression
- Total training epochs?

Clueless? Find a similar configuration in literature

How can we make things work?

- Data
- Model
- Optimizer
- Monitor
- Reproducibility

Model

- What are the **input** and **output** of your problem?
- Mindset: **tensor shapes**

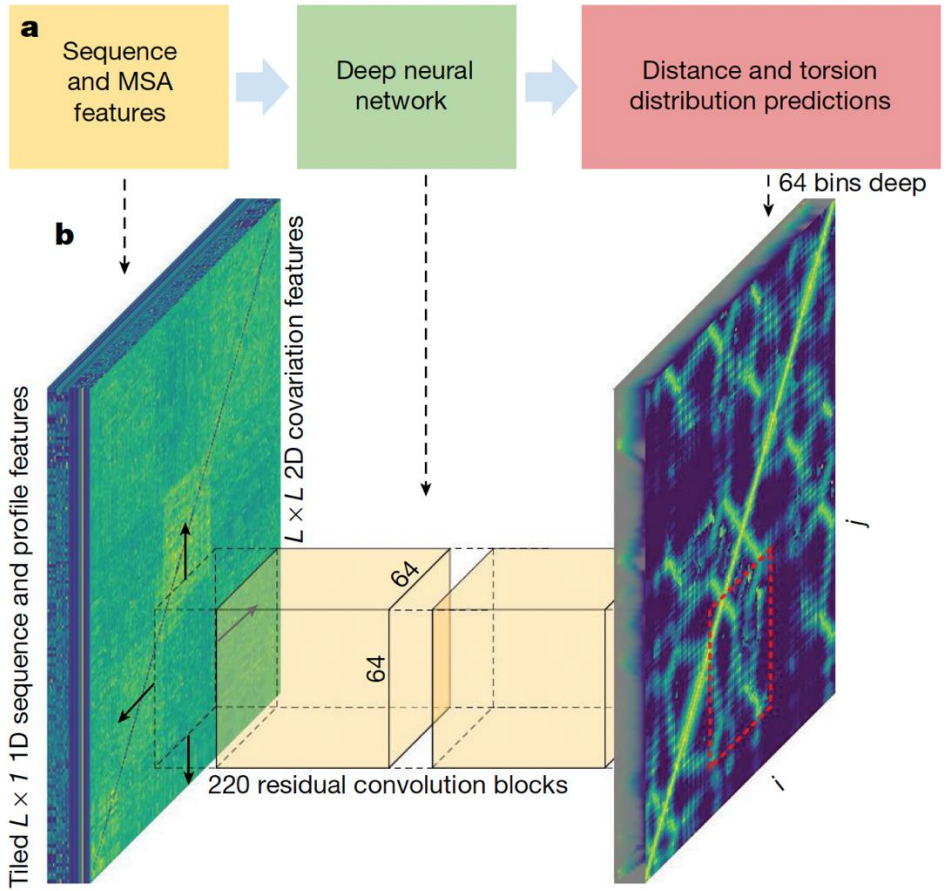
Clueless? Find a model that has the right input/output shapes!

- This is a **necessary** condition!
- A model with the right shapes might be designed for **similar** problems
- Neural networks (sometimes) are **capable** even if imperfect

“Unreasonable” yet good examples

L x L x C

- seq len: L
- 1-D tiled: L x L x c_1
- 2-D cov: L x L x c_2



L x L x 64 bins

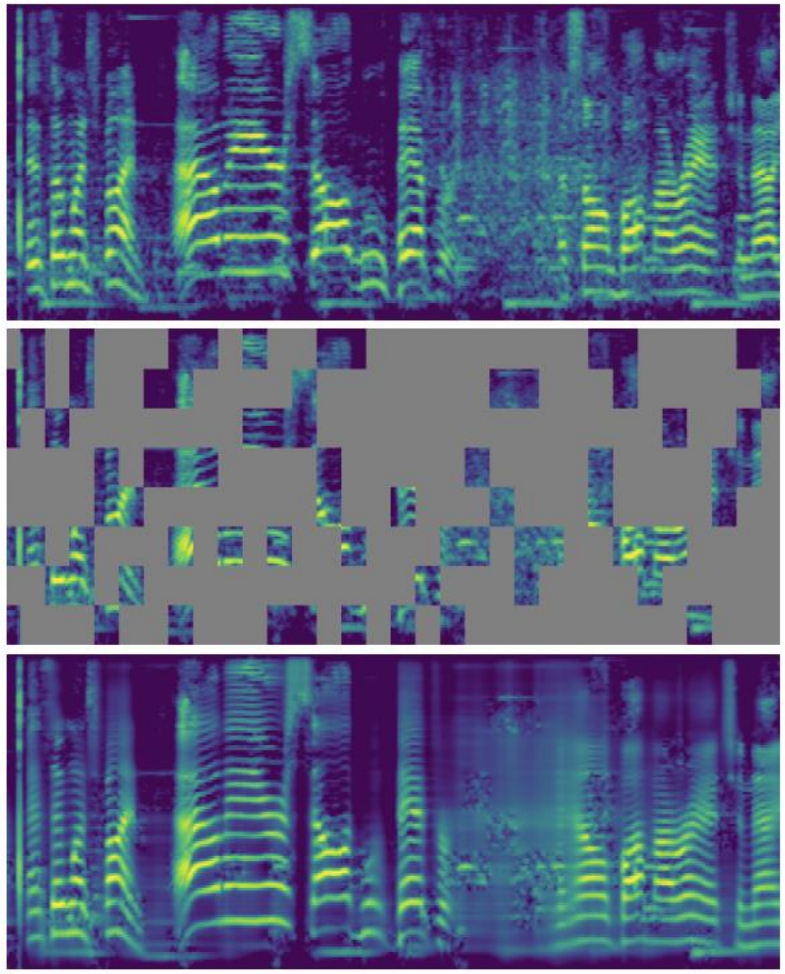
AlphaFold 1

- AlphaFold 1 is a 2D ConvNet!

Senior et al. Improved protein structure prediction using potentials from deep learning, Nature 2020

“Unreasonable” yet good examples

- Spectrogram
  - x-axis: time
  - y-axis: frequency
- Treat it like 2D images
- Apply (image-)MAE!



AudioMAE

Hunag et al. Masked Autoencoders that Listen, NeurIPS 2023

Model

Inductive biases

- 1D?
  - causal? bidirectional?
  - sequence? set?
- 2D?
  - images? spectrogram? 1D time + 1D space
  - translation-invariance? scale-invariance?
- 3D?
  - spatial 3D (x, y, z)? spatiotemporal (x, y, t)?
  - spatial/temporal translation-invariance? geometric equivariance?

Clueless? Find a similar configuration in literature

Model

Prototyping **Always start with a tiny toy model!**

- Simple MLP
  - 3 - 5 layers, 32 - 256 units, ReLU
- Simple ConvNet
  - LeNet-like: 3 - 5 conv layers, 1 - 2-layer fc (MLP) head
  - ResNet-10 or ResNet-18
- Simple Transformer
  - 3 - 5 Transformer blocks

In any case: start prototyping with reduced # channels (like 32)!

Model

Estimate your model scale **Always compute # params and FLOPs!**

- Transformers
  - # params & FLOPs are roughly proportional (assuming fixed length)
- ConvNet
  - larger feature maps: (generally) more FLOPs per parameter
- Tools to begin with:
  - torchinfo, torchsummary

Recommended: Calculate # params and FLOPs by hand

- deepen your understanding; give you direct insights

```
==========================================================================================================
Layer (type:depth-idx)          Input Shape          Output Shape         Param #            Mult-Adds
==========================================================================================================
SingleInputNet                  [7, 1, 28, 28]       [7, 10]              --                 --
├─Conv2d: 1-1                   [7, 1, 28, 28]       [7, 10, 24, 24]      260                1,048,320
├─Conv2d: 1-2                   [7, 10, 12, 12]      [7, 20, 8, 8]        5,020              2,248,960
├─Dropout2d: 1-3                [7, 20, 8, 8]        [7, 20, 8, 8]        --                 --
├─Linear: 1-4                   [7, 320]             [7, 50]              16,050             112,350
├─Linear: 1-5                   [7, 50]              [7, 10]              510                3,570
==========================================================================================================
Total params: 21,840
Trainable params: 21,840
Non-trainable params: 0
Total mult-adds (M): 3.41
==========================================================================================================
Input size (MB): 0.02
Forward/backward pass size (MB): 0.40
Params size (MB): 0.09
Estimated Total Size (MB): 0.51
==========================================================================================================
```

Example: torchinfo

https://github.com/TylerYep/torchinfo

Model

Feasibility Survey!

Always do feasibility survey when you start a project

- What scales are used **in literature**?
  - what is “standard”? what is “sota”?
- What scales **can you afford**?
  - compute budget, hardware, time
- What’s your **experimental cycle**?
  - 1 day? 1 week? 1 month?
  - **sequential**: fast turn-around (each decision depends on last results)
  - **parallelized**: breadth-first (oftentimes: hyper-parameter sweeping)

Clueless? Find a similar configuration in literature

How can we make things work?

- Data
- Model
- Optimizer
- Monitor
- Reproducibility

Optimizer

- SGD
  - work for **simple** MLP / ConvNets

- Adam
  - today's **to-go** solution
  - preferred for Transformers/**attention**
  - effortless for **prototyping**

Optimizer: hyper-parameters

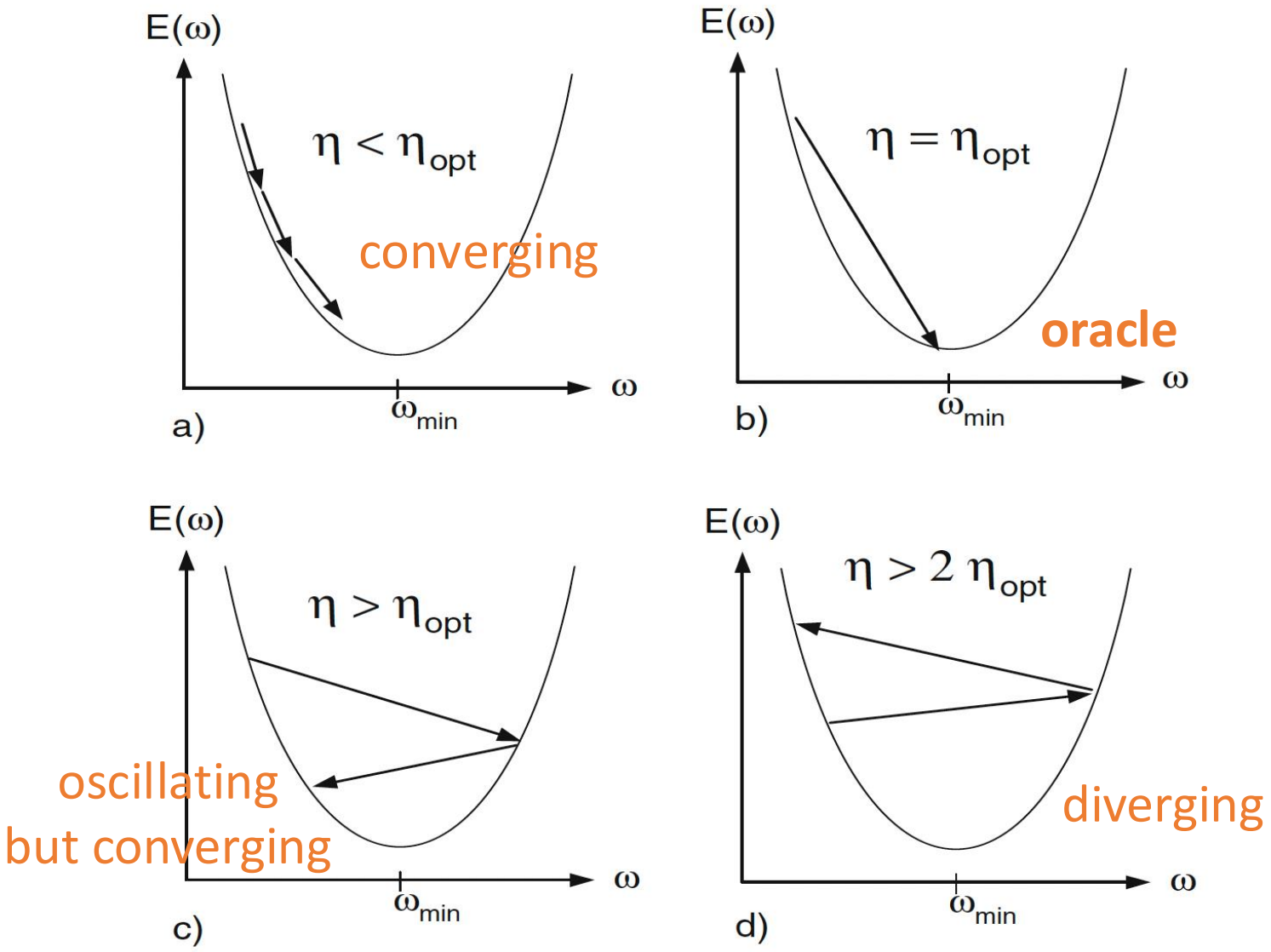
- learning rate (lr)
  - lr schedule
  - lr warmup
- batch size
- weight decay
- momentums
- initialization

Optimizer: hyper-parameters

learning rate (lr)

- “The most important hyperparameter”
- If you can tune only one thing, tune the learning rate

learning rate



LeCun et al 1998 “Efficient Backprop”

Optimizer: hyper-parameters

learning rate (lr)

- "The most important hyperparameter"

- If you can tune only one thing, tune the learning rate

Practice:

- start from some guess (e.g., **lr = 0.001**)
- search by **octave**: 2x lr or 1/2 lr (why? see previous slide)
- pick the **largest stable** lr

Optimizer: hyper-parameters

batch size

- batch size and lr are **not independent**
  - fixing lr and changing batch size **does not guarantee** similar results

- if you must change batch size, **adjust lr** accordingly
  - linear scaling (**strongly** recommended; see Goyal et al. 2017)

$$lr_{\text{new}} = lr_{\text{base}} \times \frac{B_{\text{new}}}{B_{\text{base}}}$$

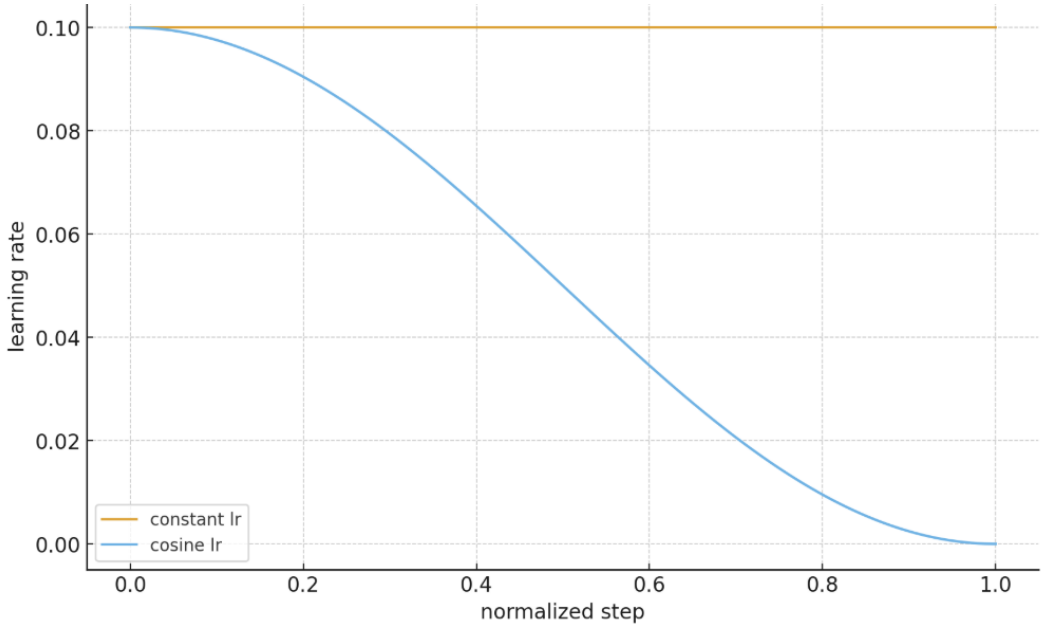
  - sqrt scaling (an alternative, worth trying)

Goyal et al. Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour, 2017

Optimizer: hyper-parameters

lr schedule

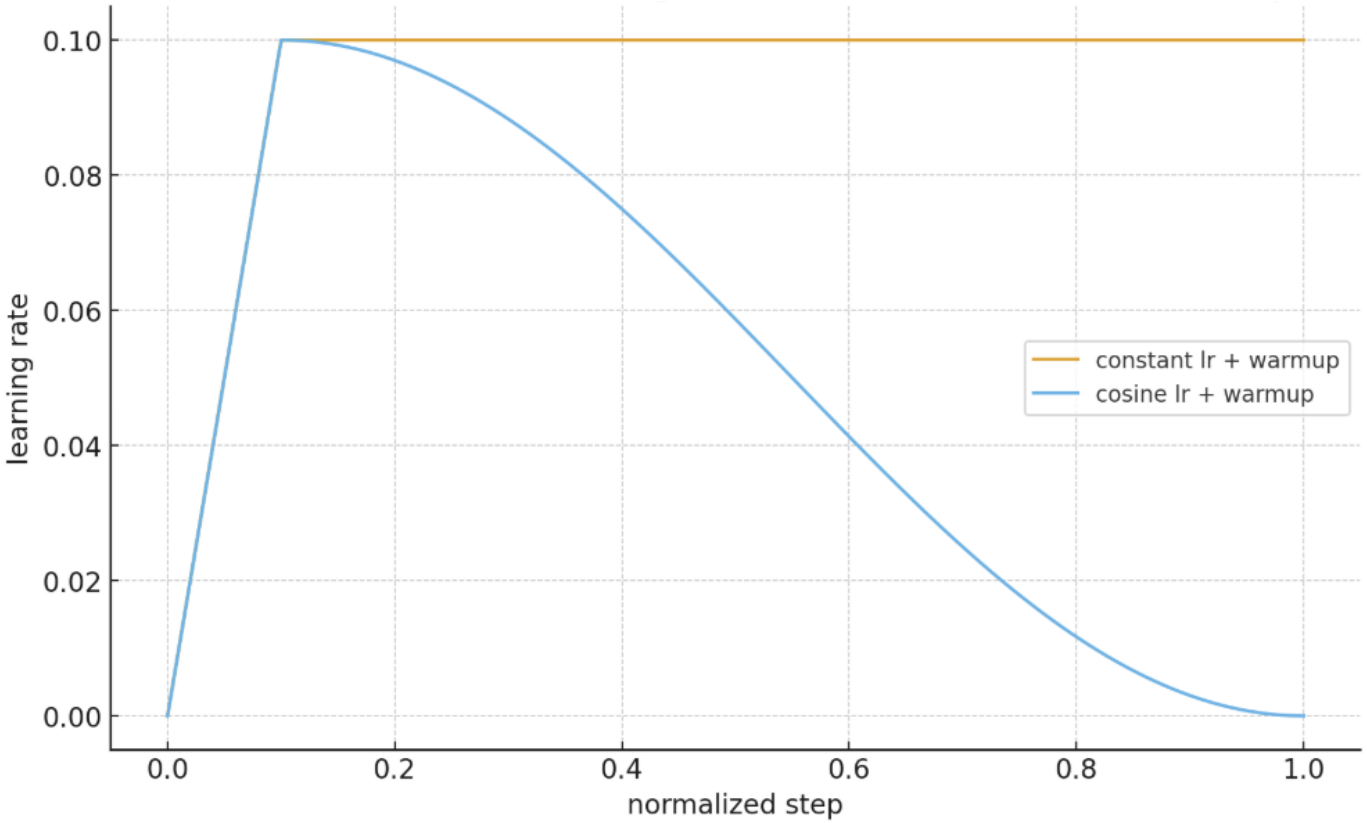
- **cosine** decay lr
  - commonly used for **recognition** problems
- **const** lr
  - commonly used for **generation** problems



Optimizer: hyper-parameters

lr warmup

- stabilized training by warm start



Goyal et al. Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour, 2017

Optimizer: hyper-parameters

weight decay (wd)

- **Classical**: L2 regularization on weights
- **Modern**: literally decay the weights (AdamW)

- **non-zero** wd: often in **recognition** (classification, detection, ...)
- **zero** wd: often in **generation**

Optimizer: hyper-parameters

momentums (beta1, beta2)

- **beta1**: first-order momentum
  - default: 0.9
  - valid for both momentum-SGD and Adam

- **beta2:** second-order momentum
  - default 0.999
  - only for Adam
  - **prototyping**: 0.95 (strongly recommended)

> We used an Adam β_2 of 0.95 instead of the default 0.999 because the latter causes loss spikes during training. We did not use weight decay because applying a small weight decay of 0.01 did not change representation quality.

from Chen et al. “Generative Pretraining from Pixels”, ICML 2020

Optimizer: hyper-parameters

Initialization

- often not viewed as part of optimizer, but **strongly** affects optimization

Common options

- Xavier / Glorot initialization
  - tanh/sigmoid/linear layers
- Kaiming / He initialization
  - ReLU or ReLU like layers (GELU, Swish, etc.)
- Gaussian with fixed std (e.g., 0.02)
  - Some Transformer models
- zero initialization
  - Some special layers: bias, exit of residual, re-init

⚠ **Be extremely careful about initialization:**
Even small deviations from a reference initialization can lead to different results.

Start to cook ...

Fit and **overfit** a tiny problem:

- **Tiny** model

- **Tiny** data
  - **single sample!** (decreasing, near-zero loss?)
  - single batch
  - tiny subset of data (100 - 1000 samples)

How can we make things work?

- Data
- Model
- Optimizer
- Monitor
- Reproducibility

Monitor

AI/DL researchers’ daily work is on coding?

Monitor

~~AI/DL researchers' daily work is on coding?~~

AI/DL researchers' daily work is on **monitoring**!

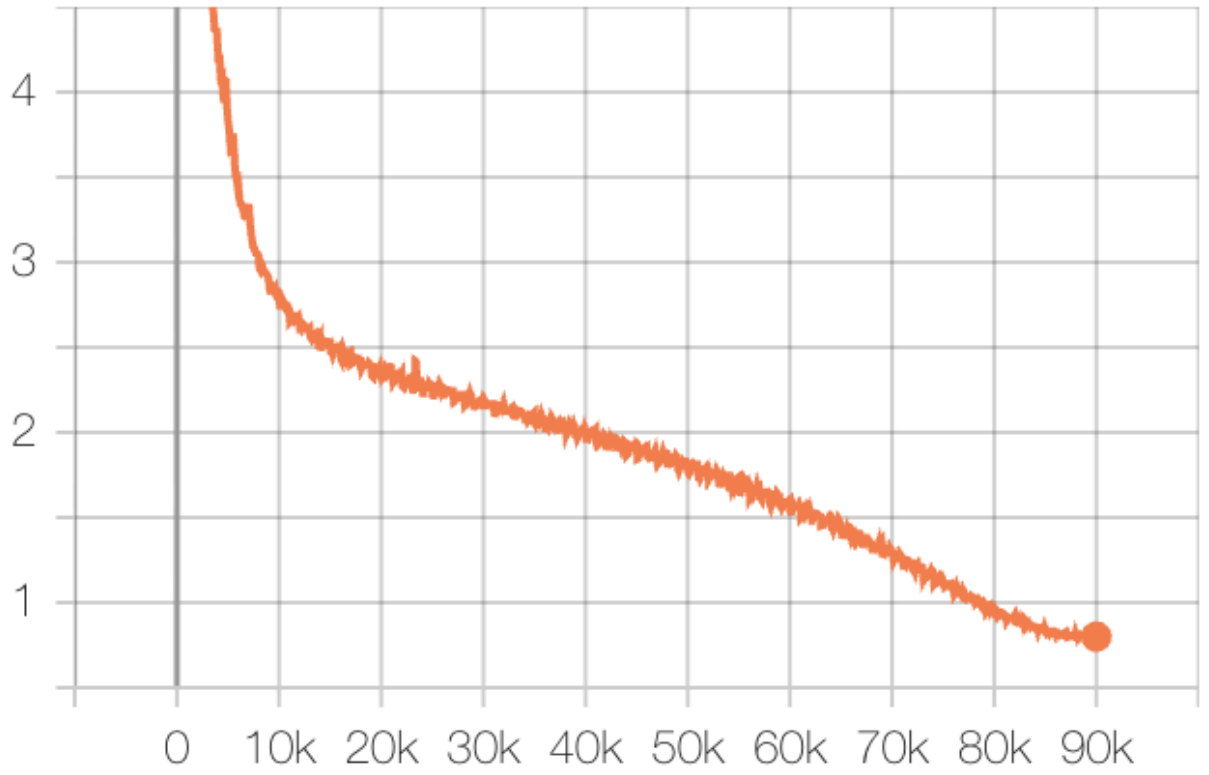
Monitor

An example “**healthy**” training loss curve (ResNet-50, ImageNet)

Practice: how to log metrics?

- x-axis: **normalized** by # images/epochs
  - calibrate different batch sizes

- average across **all devices**
- average over several **iterations**
  - e.g., 10 or 100, to reduce variance



Monitor

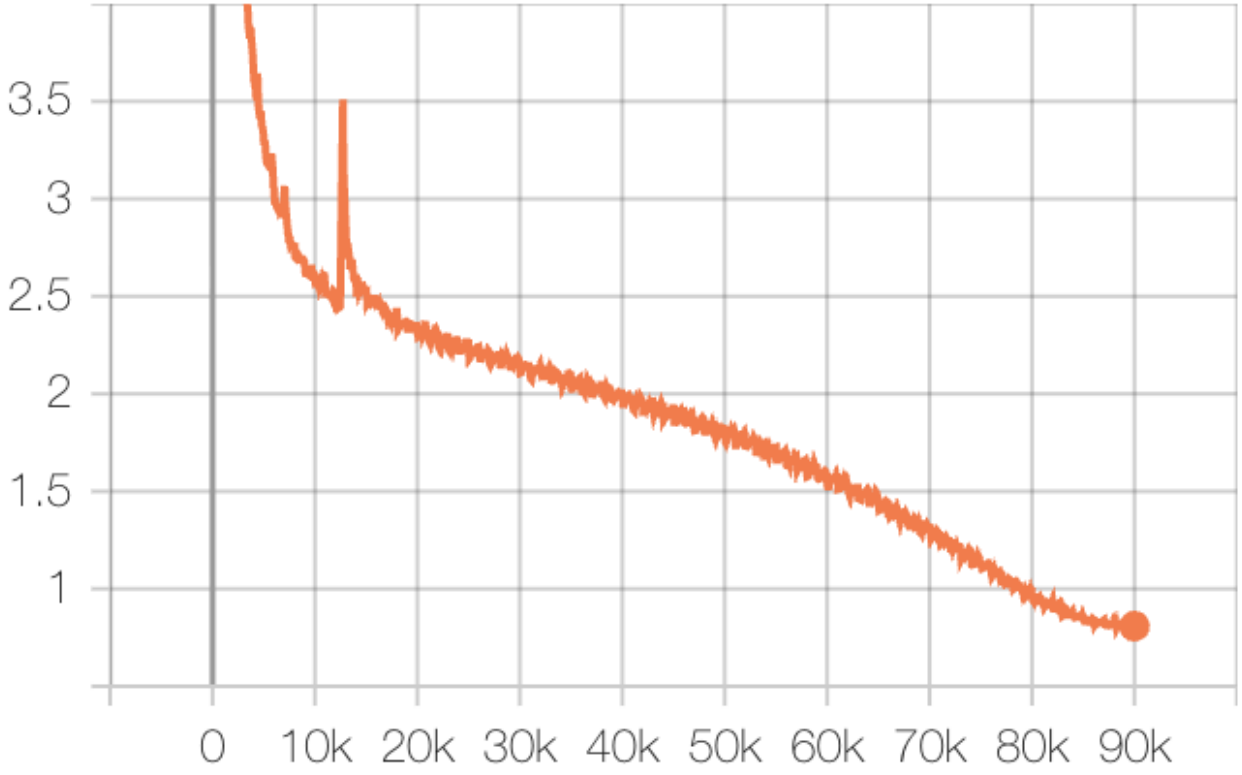
An example “**unhealthy**” training loss curve (ResNet-50, ImageNet)

Be cautious about “spikes”

- model may **not recover** from instability
- **silently** lose some capacity / units

Remedies

- reduce lr (e.g., by 2x)
- increase warmup length
- set beta2 = 0.95
- reduce batch size
- try another initialization
- **turn off fp16 / mixed-precision**



Monitor

Metrics to monitor

- **training** loss
- **validation** loss
- train/val **metrics of interest**
  - accuracy, perplexity, precision/recall, FID, ...

- **norm** of
  - gradients
  - weights
  - activations

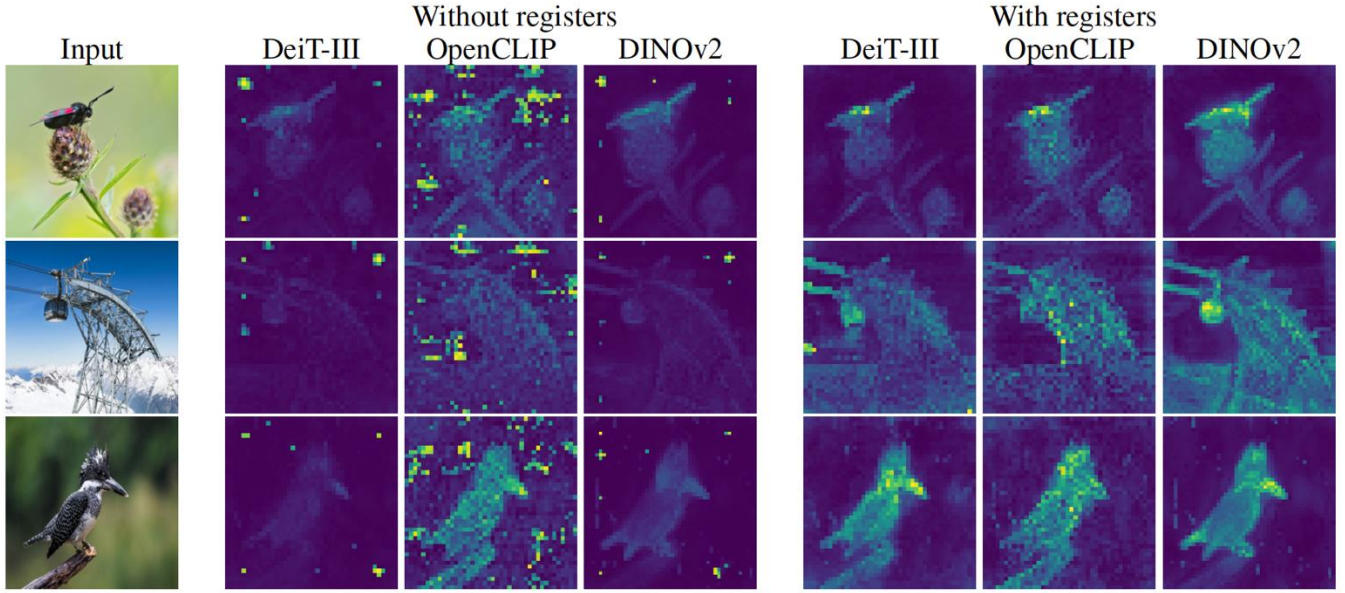
- **variance** (std) of all metrics

- **time/speed** (per iteration)

Monitor

Things to monitor

- **visualizing outputs**
  - generated images
  - restored images
  - segmentation maps ...



Darcet et al. “Vision Transformers Need Registers”, ICLR 2023

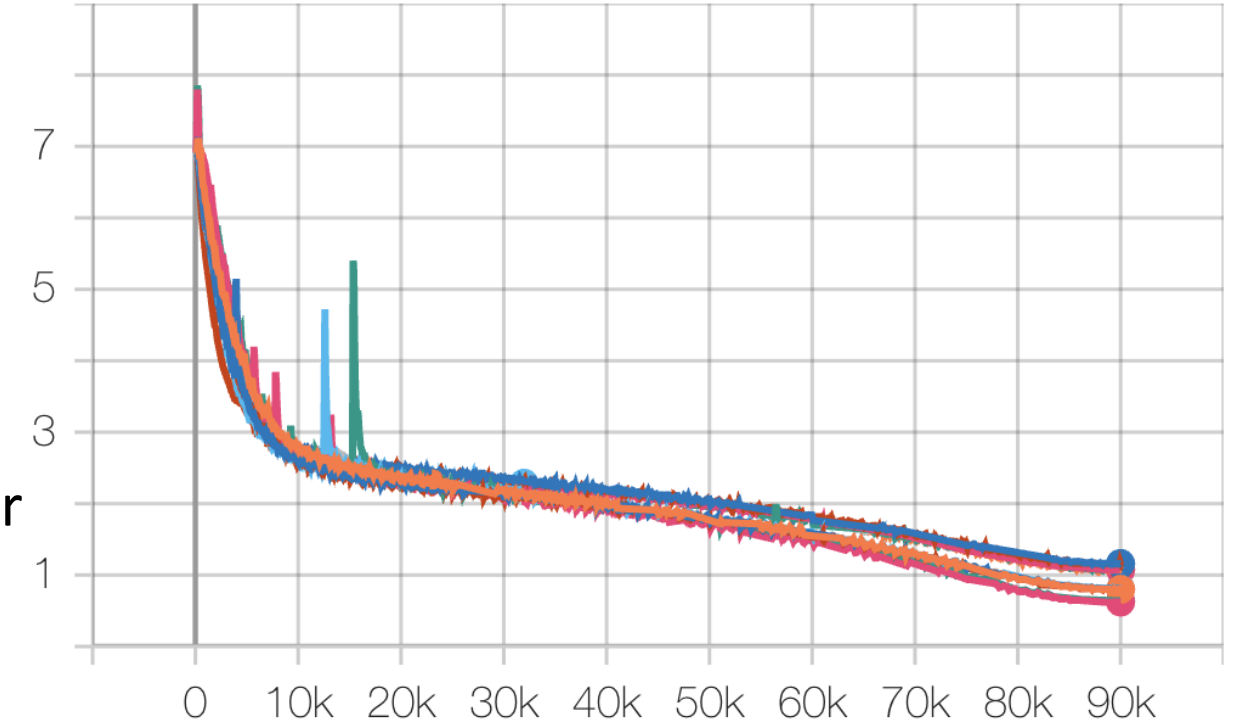
- **visualizing intermediates**
  - attention heat maps, feature maps, weight kernels

Sometimes your model is **almost** doing great, but it may fail in subtle ways that are **not visible from metrics alone**.

Monitor

Comparing different experiments

- A stand-alone experiment is often **contextless**
- Progress is revealed by comparison
- Subtle issues can be seen
  - **Overfitting**? (better train, worse val)
  - Wrong setting? (unexpected different curves)
- Stop unwanted jobs, make decisions earlier



How can we make things work?

- Data
- Model
- Optimizer
- Monitor
- Reproducibility

Reproducibility

It's amazing how reproducible DL is, given so much randomness:

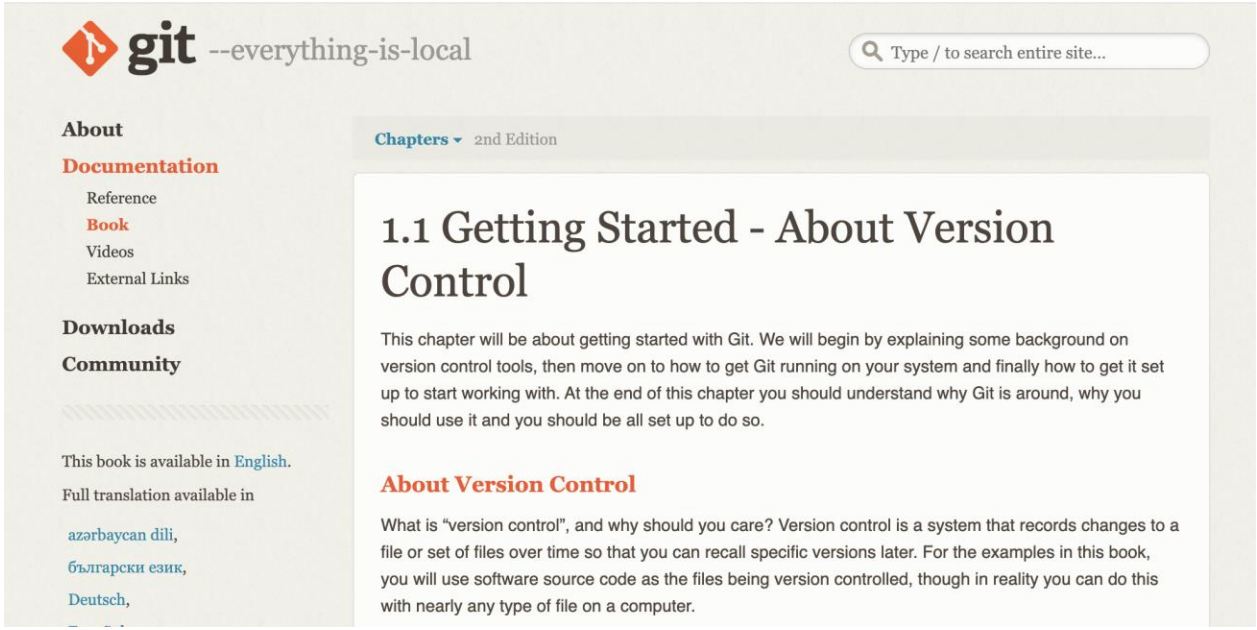
- Initialization
- Data shuffling and augmentation
- Dropout and its variants
- Generative components
- Numerical differences

But we still need to be extremely careful to make sure things go right.

Reproducibility

Manage your code:

- **Version control (Git/GitHub/Hg)**
- Commit frequently!
- Keep commits small
- Write readable commit messages

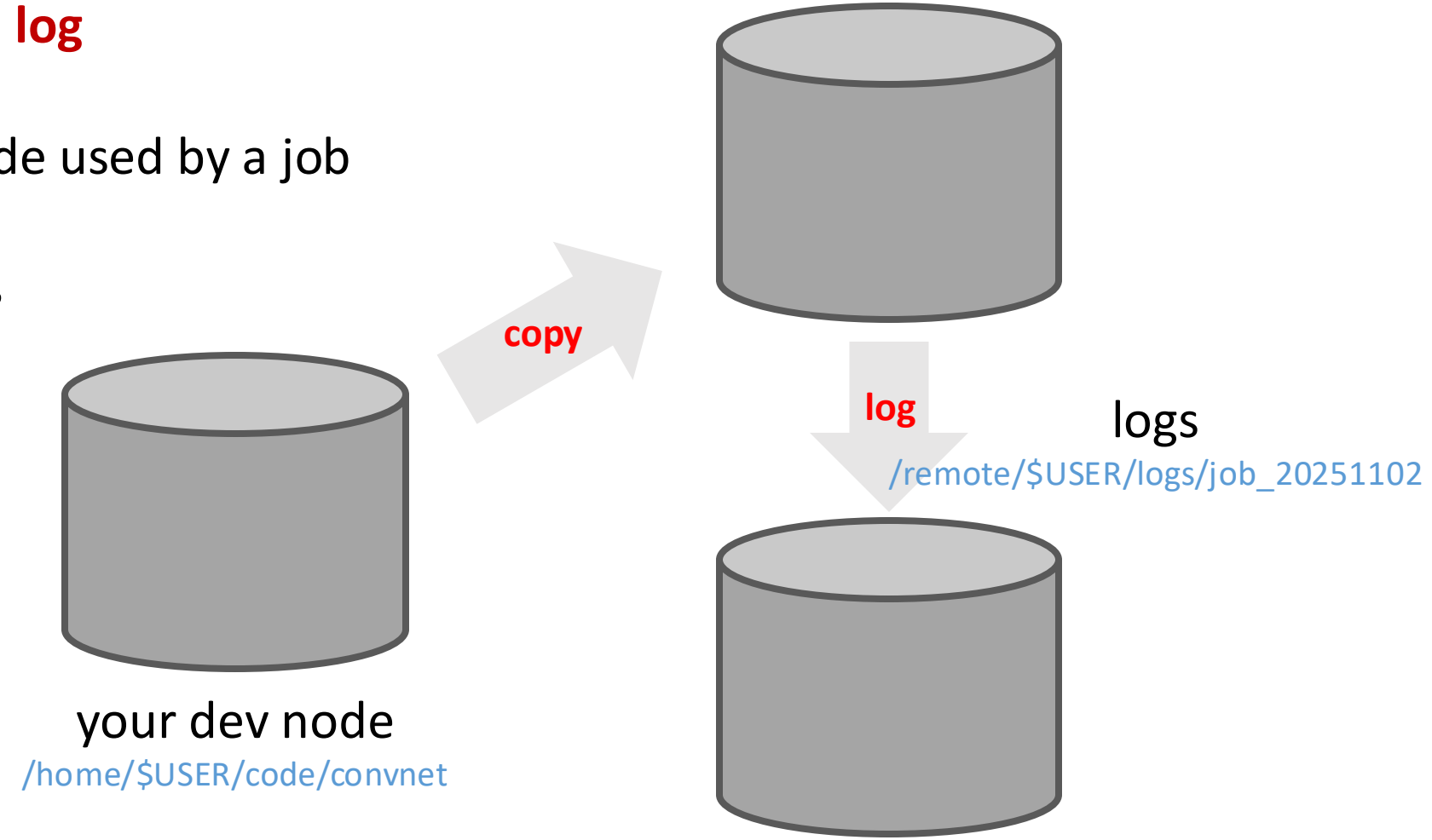


https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

Reproducibility

Manage your experiments (“jobs”)

- **One job, one code, one log**
- **Never** overwrite the code used by a job
- Log and keep **all** configs
- Practice in industry

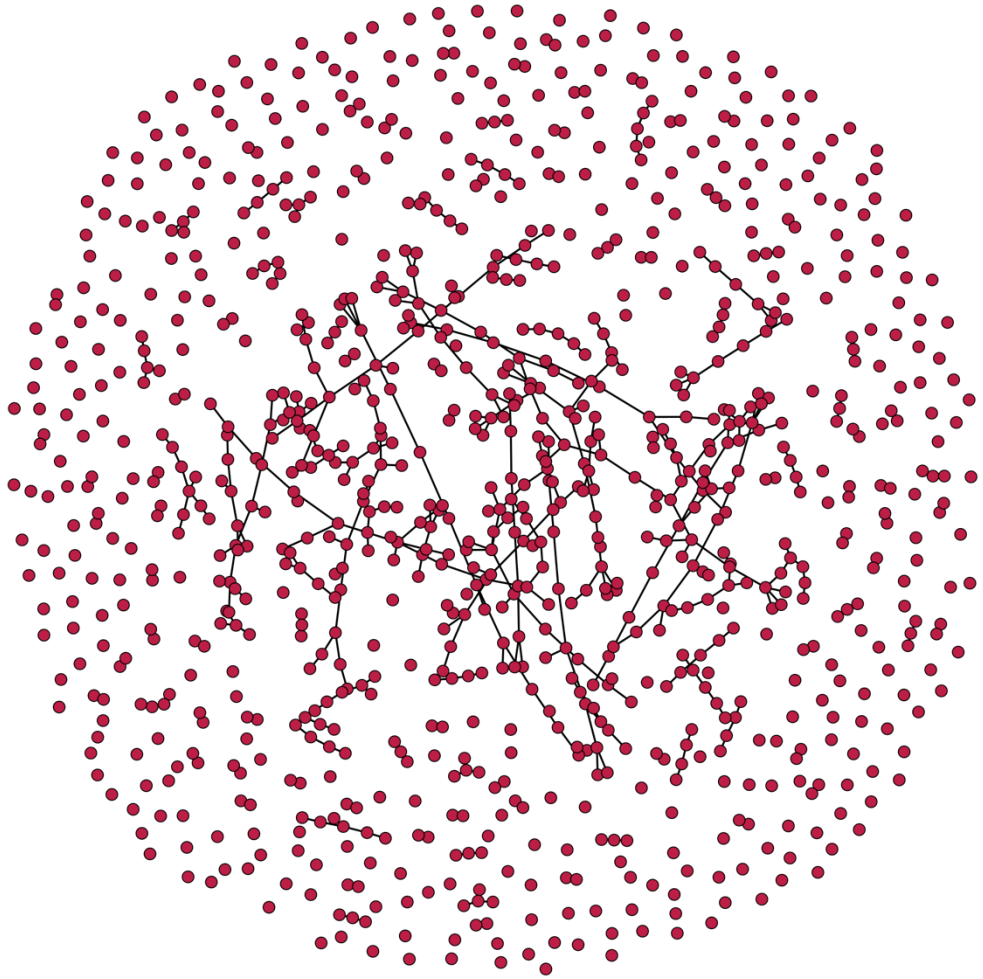


Reproducibility

Your experiments are graphs in a huge search space...

Manage your results

- **Spreadsheets**/slides/docs
- tensorboard
- wandb (a website)



How can we make things work?

- Data
- Model
- Optimizer
- Monitor
- Reproducibility

Closing Words

- Get your hands dirty!
- (You and your model): learn by exploring
- Stand on the shoulders of giants
  - Clueless? Find a similar scenario in the literature
- Go build something cool!